

Backend

```
script.js
1 // Fundamentals of JavaScript:
2 // arrays and objects
3 // functions return
4 // async js coding
5
6 // line by line code chale isey kahte hai synchronous
7 // jo bhi code async nature ka ho, usey side stack mein bhej do and
  agle code ko chalao jo bhi sync nature ka ho, jab bhi saara syn code
  chal jaaye, tab check karo ki async code complete hua ya nahi and
  agar wo complete hua ho to usey main stack mein laao and chalo
```

```
4 // async js coding
5
6 async function abcd(){
7     var blob = await fetch(`https://randomuser.me/api/`);
8     var ans = blob.json();
9
10    console.log(ans);
11 }
12
13 abcd();
```

```
4 // async js coding
5
6 async function abcd(){
7     var blob = await fetch(`https://randomuser.me/api/`);
8     var ans = await blob.json();
9
10    console.log(ans.results[0].name);
11 }
12
13 abcd();
```

```
1 js se backend nahi ban sakta
2 kyuki js ke paas wo functionalities nahi hai jinse backend banta hai
3 ryan dahl isne socha js se backend banna chahiye
4 google chrome ka v8 engine ka code open source hai and ryan dahl us code mein
  chhedkhaani kar raha hai
5 kyuki chrome ka v8 engine bana hai c++ mein
6 fir usne socha ki humein to js mein code karna hai
```

```
1 hum js ka code likhege jo ki wrapper layer of js receive karegi and wo code v8
  engine ke c++ modules ke saath ek server create krega
```

```
1 const fs = require('fs');
2
3 fs.writeFile("hey.txt", "hey hello kaise ho", function(err){
4   if(err) console.error(err);
5   else console.log("done");
6 })
```

Node.js v21.7.2 | [Table of contents](#) | [Index](#) | [Other versions](#) | [Options](#)

fs.appendFile(path, data[, options], callback)

► History

```
0); script.js > ...
1 const fs =require('fs');
2
3 fs.rename("hey.txt","hello.txt",function(err){
4   if(err) console.error(err);
5   else console.log("done");
6 })
```

```
() : script.js > ...
1  const fs =require('fs');
2
3  fs.copyFile("hey.txt","./copy/chacha.txt",function(err){
4    if(err) console.error(err);
5    else console.log("done");
6  })
```

```
backend\
() : script.js x hello.txt
() : script.js > ...
1  const fs =require('fs');
2
3  fs.unlink("hello.txt",function(err){
4    if(err) console.error(err);
5    else console.log("removed");
6  })
```

```
Go Run Terminal Window Help
youtubebackend
script.js x npm init -> package.json -> lekha jokha Untitled-1
script.js > recursive
1  const fs = require('fs');
2
3  fs.rmdir("./copy", {recursive: true}, function(err){
4    if(err) console.error(err);
5    else console.log("removed");
6  })
```

```
Internet par kuchh bhi karne ke liye rul Untitled-1
1  internet par kuchh bhi karne ke liye rules banaaye gaye hai unke
   dwaara jinhone internet banaaya hai, ab un rules ko follow karna
   jaruri hai, aur ye rules follow ho isliye ye rules aap ke operating
   system ke software mein pre installed aate hai
```

{}; script.js > ...

```
1  const http=require('http');
2
3  const server = http.createServer(function(req,res){
4      res.end("hello world");
5  })
6
7  server.listen(3000);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

kartiksuryavanshi@Kartiks-MacBook-Air backend\ % node script.js
□

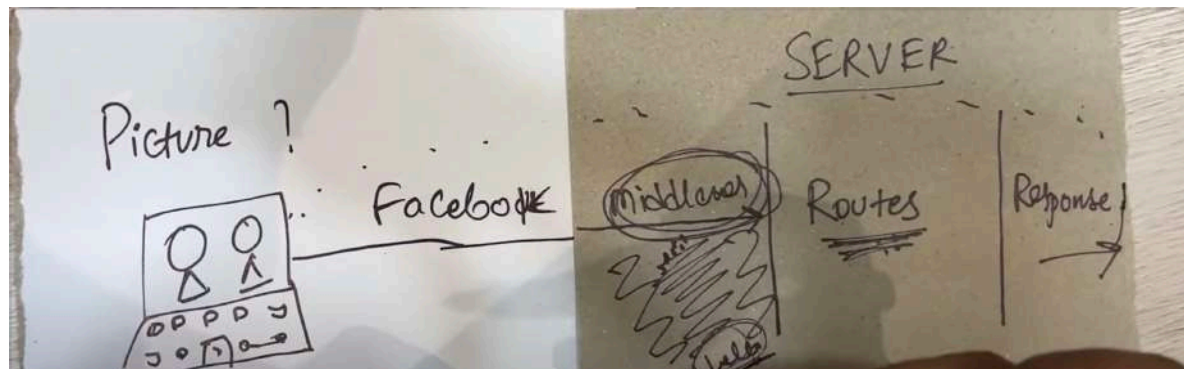
```
script.js x
script.js
1  // NPM understanding
2
3  // installing and uninstalling anything basics & advanced
4  // install -> npm i packageKaNaam
5  // uninstall -> npm uninstall packageKaNaam
6  // installing particular versions
7
8  // understanding node_modules
9
10 // dependencies - packages and packages ki dependencies
11 // devdependencies - aise packages jo sirf development mei
    aayege par jab app ban jaayega aur upload ho jaayega tab
    packages ko use nahi kar rahe honge
12
13 // scripts - understanding default scripts PATH and custom
```


Backend (Node JS) Series - Learn What Matters 2: Mastering Node.js Fundamentals

middleware ek aisa function hai jo kisi bhi route se pahle chalta hai, jiska matlab aapka route chalne se pahle agar aap koi kaam karana chaahte ho to middleware ka upyog kiya jaa skta hai

- 1
- 2
- 3 route pe chalne se pahle print karo chala on console
- 4 route chalne se pahle route pe hits counter ki value badho

1:42:35 / 1:48:38



node vs express - node is the main thing express ke code se hum server ka code likh pate and server kaisa react karega wo bhi express ke help se likhte hai

what is express js - package, routing

why express js - http is difficult to use, express makes this easier

routing - routes banaane ke process ko hum kahte hai routing.

- /profile
- /home
- /contact
- /profile/harsh/iuyahyicuhaihkcaljc9acuas/u7catsi7cuah8cuajc

middleware

```
1 node ke saath ek dikkat hai ki agar control ek baar bhi kisi  
middleware par gaya to control khud se agle route/middleware  
par nahi jaayega, use agle par lejaane ke liye aapko push  
karna padega aur ye push kahlaayega next ko chalaana
```

```
3 middleware  
4  
5 request and response, next  
6 req mein saara data hota hai aane waale user ki request ki  
taraf ka, jaise ki uski location, device info and other  
things, res mein controls hote hai jinke basis pe hum server  
se response bhej paate hai, next is just a push so that our  
request moves to the next thing which should be executed  
7
```

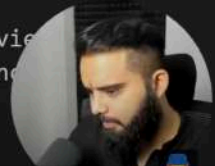
```
1 url mein jab bhi aapke paas ek aisa pattern ho  
2  
3 /profile:username  
           ↖  
         Params
```

```
- to make any route dynamic you can use : at the place where  
you want to make it dynamic, and to access there value use  
req.params  
/author/books/issued/harsh  
/author/books/issued/harshita  
/author/books/issued/harshit  
/author/books/issued/:username
```

```

1 ejs setup karne ke liye
2
3 1) ejs install
4 npm i ejs
5
6 2) configure ejs
7
8 app.set("view engine", "ejs");
9
10 3) ek views folder banao
11
12 4) usmein ejs files banao
13
14 5) send ki jagah render karo => render karte waqt make sure aap view
    folder ke andar waali hi koi file kaa naam likhein, aur render function
    ejs mention naa karein

```



you want to make it dynamic, and to access there value use
req.params

```

20
21 template engines => ejs is very very similar to html
22
23 static files

```

```

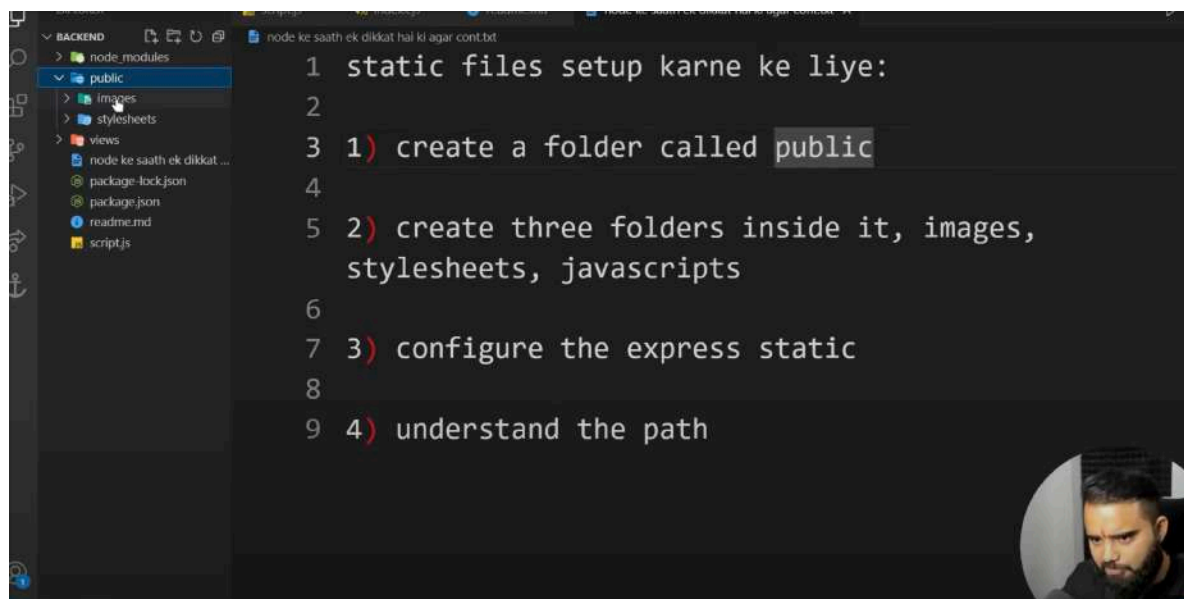
5
6 app.get("/", function (req, res) {
7   res.render("index", {age:12});
8 });
9
10 app.get("/contact", function (req, res) {
11   res.render("contact", {name: "harsh"});
12 });

```

```

<h2 class="text-2xl font-bold md:text-4xl
and:leading-tight dark:text-white">Meet the <%= name %></
h2>

```

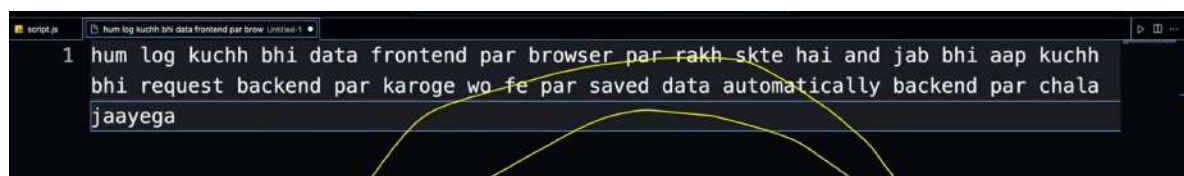


```
1 static files setup karne ke liye:
2
3 1) create a folder called public
4
5 2) create three folders inside it, images,
  stylesheets, javascripts
6
7 3) configure the express static
8
9 4) understand the path
```

```
21 template engines => ek markup style jo ki baad mein convert
    hojaayegi html mein
22
23 static files
24 images, stylesheets, frontend js setup karna
25
26 http methods - get post
27
28 error handling
```

Cookies

```
1 form handling and working with the forms
2 handle backend process of forms and making sure the data coming from any frontend
  lib, fw, templating engines, we still handle it at the backend process
```



```
1 hum log kuchh bhi data frontend par browser par rakh skte hai and jab bhi aap kuchh
  bhi request backend par karoge wo fe par saved data automatically backend par chala
  jaayega
```

```
tumne to bheja tha plain text par server ko mila blob which is not directly readable
ab is cheej ko hanld karna padega ki hum us blob ko waapas se readable kar sake
```



```
Setting up public static files  Untitled-1  ●  index.js  X
index.js
1 console.log(__dirname+'/public');
2
3
4
5
6
7 // const express = require('express');
8 // const app = express();
9
10 // app.use(express.json());
11 // app.use(express.urlencoded({ extended: true }));
12 // app.use(express.static(path.resolve(__dirname, 'public')));

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Users/async/Desktop/youtubebackendpublic
[nodemon] clean exit - waiting for changes before restart
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Users/async/Desktop/youtubebackend/public
[nodemon] clean exit - waiting for changes before restart
```

```
9
10 app.get("/", function(req, res) {
11     res.render("index");
12 });
13
14 app.get("/profile/:username", function(req, res) {
15     res.send(`welcome, ${req.params.username}`);
16 });
17
18 app.get("/author/:username/:age", function(req, res) {
19     res.send(`welcome, ${req.params.username} of age ${req.params.age}`);
20 });
21
22 app.listen(3000, function() {
```

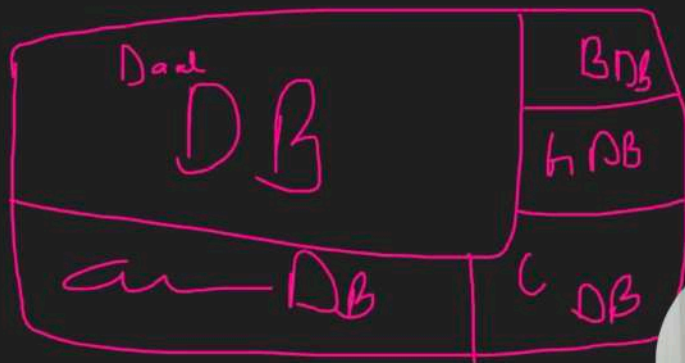
1 // hum logo ko kaafi saara kaam karna padta hai express setup krne mein iska matlab har baar jab hum naya project banaayege to har baar fir se utna hi kaam krna padega, matlab ki poora code likho and sab kuchh setup karo, is time ko bacha sakte ho with help of express generator, use express generator and it will make the folder structure for you and it will also write the basic code for the project

criptjs

```
2 // sabse pahle jeevan mein ek baar laptop par install
  karo globally
3 // npm i express-generator -g
4
5 // to create new app anywhere:
6 // open cmd move to desktop
7 // create new app :
8 // express appname --view=ejs
9
10 // now use two commands
11 // cd appname
12 // npm i
13 // open it on vs code
```



har naye app ka data store hoga storage mein, par usey directly rakne ki jagah ek container mein rakhege, us container mein sirf us app ka data aayega



```
1 // schema (code) => documents (db)
2
3 // ek app ka poora data => db
4 // ek app mein variety of data hota hai par poora data hota app ka hi
  hai, par us data ka sub category kehlaata hai collection
5 // collection matlab ki bola users ka data, ek user pe baat kri to hua
  document
```

```
edit Selection View Go Run ... lolopopo
users.js x
tes > users.js
1 // install mongodb
2 // install mongoosejs
3 // require and setup connection
4 // make schema
5 // create model and export
I
```

```
index.js router.get('/delete') callback
1 var express = require("express");
2 var router = express.Router();
3 const userModel = require("./users");
4
5 router.get("/", function (req, res) {
6   res.render("index");
7 });
8
9 router.get("/delete", async function (req, res) {
10   let deleteduser = await userModel.findOneAndDelete({
11     username: "harsh"
12   });
13   res.send(deleteduser);
14 });
15
16 module.exports = router;
17
Live Share Lin 13 Col 25 Spaces 2 UI-8 11 JavaScript
```

```

25 Mongoose?
25 How can I filter documents based on a specific field's length in
25 Mongoose?
26 session and cookies
27 understanding sessions
28 creating sessions, using sessions accross routes and destroying
28 sessions
29
30 creating cookies, using cookies accross routes and destroying
30 cookies
31
32
```

```
index.js x app.js users.js readme.md session: Untitled-1
routes > index.js > router.get("/removeban") callback > req.session.destroy() callback
4 router.get("/", function (req, res) {
5   req.session.ban = true;
6   res.render("index");
7 });
8
9 router.get("/checkban", function (req, res) {
10   if (req.session.ban === true) {
11     res.send("you are banned");
12   }
13 });
14
15 router.get("/removeban", function (req, res) {
16   req.session.destroy(function(err){
17     console.log(err);
18     res.send("ban removed");
19   })
20 });
```

```
index.js x app.js users.js readme.md create Untitled-1
1 create
2 req.session.koibhinaam = koibhivalue;
3
4 read
5 req.session.koibhinaam
6
7 delete
8 req.session.destroy
```

```
routes > index.js > router.get("/delete") callback
2 var router = express.Router();
3
4 router.get("/", function (req, res) {
5   res.cookie("age", 25);
6   res.render("index");
7 });
8
9 router.get("/read", function (req, res) {
10   console.log(req.cookies.age);
11   res.send("check");
12 });
13
14 router.get("/delete", function (req, res) {
15   res.clearCookie("age");
16   res.send("clear hogayi");
17 });
18
19 module.exports = router;
```



```

1 // cookie setup
2 // res.cookie("name", value);
3
4 // cookie reading
5 // req.cookies.name
6
7 // cookie delete
8 // res.clearCookie("name")

```

1 jab bhi aap kisi ejs page ko dekhege waha par aapko kisi prakaar ka koi information dena hai, wo kehlata hai flash messages, they are more like good looking alerts, warning and descriptions

```

1 install connect-flash
2 make sure you setup express-session
3 make sure you put connect flash in a app.use function
4 kisi bhi route mein aap ko flash create karna hai
5 kisi bhi doosre route par app use chalane ka try karein
6
7 AAP CONNECT FLASH KO USE NAHI KR SKTE BINA EXPRESS SESSION

```

```

8 router.get('/login', function(req, res) {
9   // agar login hojaaye to login page ke baad profile page dikhaado
10  // agar naa ho to fir, mujhe is route se kisi aur route lejao jaise
    ki /error and waha par dikhao failed
11 });
12

```

```

7
8 router.get('/login', function(req, res) {
9   // flash message aapko ye allow krte hai ki aap is route mein bane
    huye data ko doosre route mein use kr sko
10 });

```

```
Backend - The EndGame (Part-2) | ElevateYour Skills | Backend (Node JS) Series

install these packages: Untitled-1
1 install these packages:
2   npm i passport passport-local passport-local-mongoose mongoose
   express-session
3
4 write app.js code first in app.js file and write it after view engine
  and before logger
5
6 setup users.js then properly
7 |
8 in index.js try register first and then other codes as well
```

```
user ko profile dene se pahle ye check k Untitled-1
1 user ko profile dene se pahle ye check karna hota hai
  ke user hai kaun - authentication
2
3 authorized to perform some tasks which are only
  allowed to be executed by the admin
```

```
er har baar bhool jata hai ki aap ka Untitled-1
1 server har baar bhool jata hai ki aap kaun ho, ab
  kyuki server bhool jaata hai aap kaun ho to har baar
  wo har kaam ke liye aapse puchhega aap kaun ho
```

```
ahle humein kuchh cheejein seekhni hai Untitled-1
sabse pahle humein kuchh cheejein seekhni hai alag alag
i) cookie kaise set karte hai
ii) bcrypt kaise use karte hai for password encryption and decryption
iii) jwt kya hai and jwt mein data kaise store karein and baahar nikaalein
```

```
1  const express = require ('express');
2  const app =express();
3  const bcrypt =require('bcrypt');
4
5
6
7  app.get("/",function(req,res){
8      bcrypt.genSalt(10, function(err, salt) {
9          bcrypt.hash(kartik, salt, function(err, hash) {
10              console.log(hash);
11          });
12      });
13  })
14
15
16
17  app.listen(3000);
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS + v ... ^

```
208:32)
    at Module._load (node:internal/modules/cjs/loader:
1024:12)
```

node
node

To check a password:

```
// Load hash from your password DB.
bcrypt.compare(myPlaintextPassword, hash, function(err, result) {
    // result == true
});
bcrypt.compare(someOtherPlaintextPassword, hash, function(err, result) {
    // result == false
});
```

```
app.js x <% index.ejs user.js create user account Untitled-1 •
app.js > [e] jwt
1  const express = require('express');
2  const app = express();
3  const userModel = require("../models/user");
4  const bcrypt = require('bcrypt');
5  const jwt = require('jsonwebtoken');
6
7  const cookieParser = require('cookie-parser');
8  const path = require('path');
9
10 app.set("view engine", "ejs");
11 app.use(express.json());
12 app.use(express.urlencoded({ extended: true }));
13 app.use(express.static(path.join(__dirname, 'public')));
14 app.use(cookieParser());
15
16 app.get('/', (req, res) => {
17   res.render('index');
18 });
19
20 app.post('/create', (req, res) => {
21   let {username, email, password, age} = req.body;
```

```
app.post('/create') callback > bcrypt.genSalt() callback > bcrypt.hash() callback
app.post('/create', (req, res) => {
  let {username, email, password, age} = req.body;

  bcrypt.genSalt(10, (err, salt) => {
    bcrypt.hash(password, salt, async (err, hash) => {
      let createdUser = await userModel.create({
        username,
        email,
        password: hash,
        age
      });

      let token = jwt.sign({email}, "shhhhhhhhhhhhh");
      res.cookie("token", token);
      res.send(createdUser);
    })
  })
});
```


Authentication & Authorization | JWT Token | MongoDB | Part 15 - Backend Developr

```
app.js x login.ejs user.js create user account Untitled-1
app.js > app.post("/login") callback > bcrypt.compare() callback
42     res.render('login');
43   });
44
45   app.post("/login", async function (req, res) {
46     let user = await userModel.findOne({email: req.body.email});
47     if(!user) return res.send("something is wrong");
48
49     bcrypt.compare(req.body.password, user.password, function (err, result) {
50       if(result){
51         let token = jwt.sign({email: user.email}, "shhhhhhhhhhh");
52         res.cookie("token", token);
53         res.send("yes you can login");
54       }
55       else res.send("something is wrong");
56     })
57   });
58
59   app.get("/logout", function(req, res){--
62   })
63
64   app.listen(3000);
```

Ln 51, Col 43 (8 selected) Spaces: 4 UTF-8 LF ()

36:57 / 38:03 • Making Login Route >

```
app.js index.ejs create user account Untitled-1
1 create user account
2   mongoose
3   schema
4   model
5   usercreate -> password -> hash
6   jwt token -> cookie
7
8   login -> token -> decrypt -> email
```

Data Association | MongoDB | Part 16 - Backend Development

```
EXPLORER
  YOUTUBEBACKEND
    models
      post.js
      user.js
    node_modules
    app.js
    package-lock.json
    package.json

models > user.js > userSchema
1  const mongoose = require('mongoose');
2
3  mongoose.connect("mongodb://127.0.0.1:27017/testingthetdatabase");
4
5  const userSchema = mongoose.Schema({
6    username: String,
7    email: String,
8    age: Number,
9    posts: [
10     {type: mongoose.Schema.Types.ObjectId}
11   ]
12 })
13
14 module.exports = mongoose.model('user', userSchema);
```

```
package.json  app.js  user.js  post.js
models > post.js > postSchema > user > ref
1  const mongoose = require('mongoose');
2
3  const postSchema = mongoose.Schema({
4    postdata: String,
5    user: {
6      type: mongoose.Schema.Types.ObjectId,
7      ref: "user"
8    },
9    date: {
10     type: Date,
11     default: Date.now
12   }
13 })
14
15 module.exports = mongoose.model('post', postSchema);
```

```
9
10 > app.get("/create", async function (req, res) {...
18 })
19
20 app.get("/post/create", async function (req, res) {
21   let post = await postModel.create({
22     postdata: "hello saare log kaise ho",
23     user: "662f4213fb0db6916e936dcc"
24   })
25
26   let user = await userModel.findOne({_id: "662f4213fb0db6916e936dcc"});
27   user.posts.push(post._id);
28   await user.save();
29   res.send({post, user});
30 })
31
32 app.listen(3000);
```

Ln 29, Col 26 Spaces: 4 UTF-8 LF () Java

47 / 22:22 Explaining the code

```
users post likh paayegi
login and register
logout
post creation
post like
post delete
```

mongodb+srv://kartik2005:kartik2005@cluster0.k6ltr.mongodb.net/instaclone?
retryWrites=true&w=majority&appName=Cluster0"